



SENAI

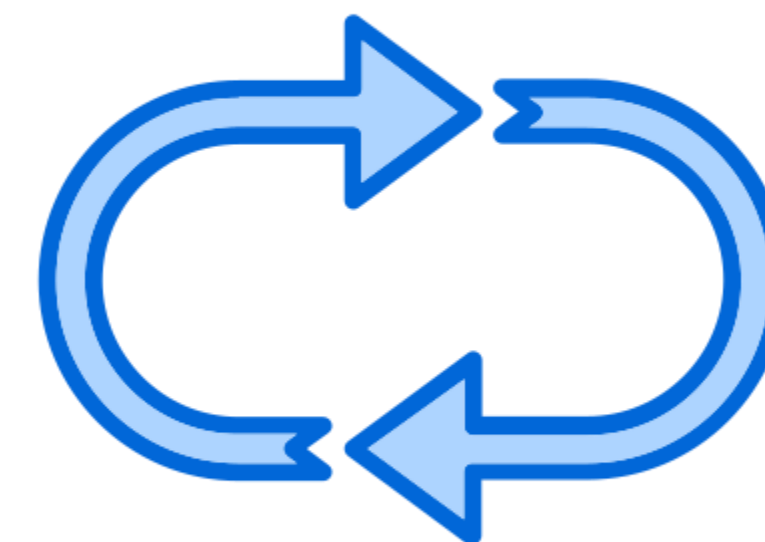
Lógica de programação

Florisvaldo Junior

Estruturas de repetição



Estruturas de repetição são estruturas utilizadas em algoritmos e programas para **executar um mesmo conjunto de instruções várias vezes**, enquanto uma **condição for verdadeira** ou até que uma **condição seja satisfeita**.

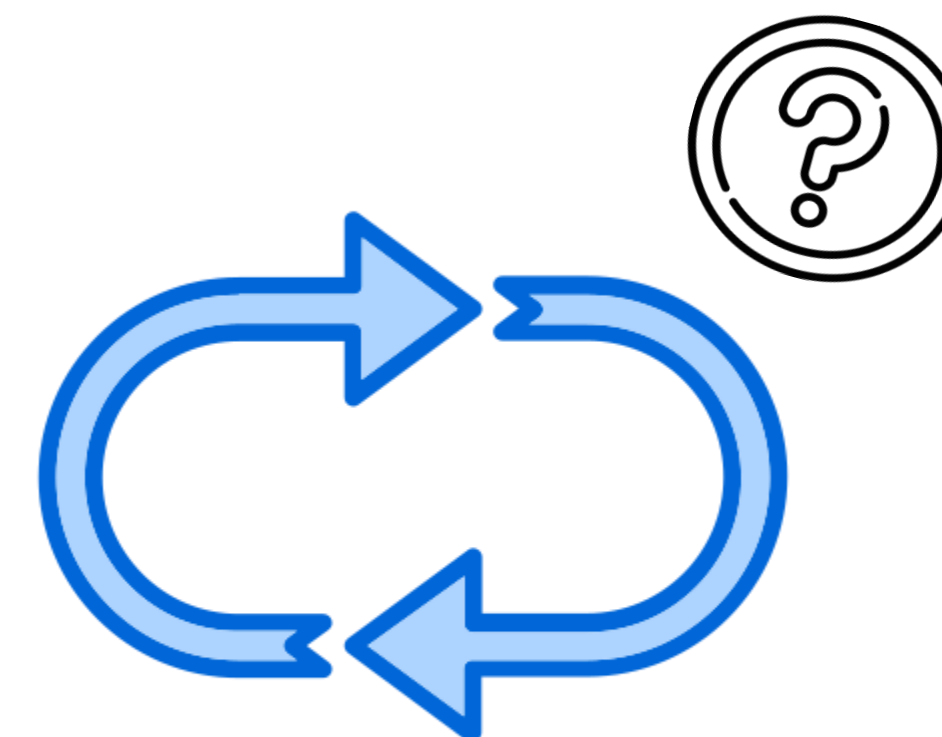


Por que usar estruturas de dados?



As estruturas de repetição resolvem um problema crítico de **escalabilidade**. Quando um bloco de código precisa ser executado diversas vezes, escrevê-lo de forma manual e fixa torna o sistema rígido e ineficiente.

Se exibir valores de 0 a 100 sem loops já é um processo trabalhoso e propenso a erros, realizar a mesma tarefa para 1.000 ou 10.000 registros seria inviável. As repetições trazem a **flexibilidade** necessária para lidar com qualquer volume de dados com poucas linhas de código.



Comparação

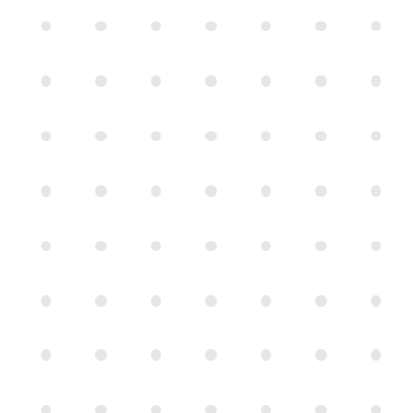


Sem estrutura de repetição

- Início;
- Escrever "número 1";
- Escrever "número 2";
- Escrever "número 3";
- Escrever "número 4";
- Escrever "número 5";
-
- Escrever "número 1000";
- Fim

Com estrutura de repetição

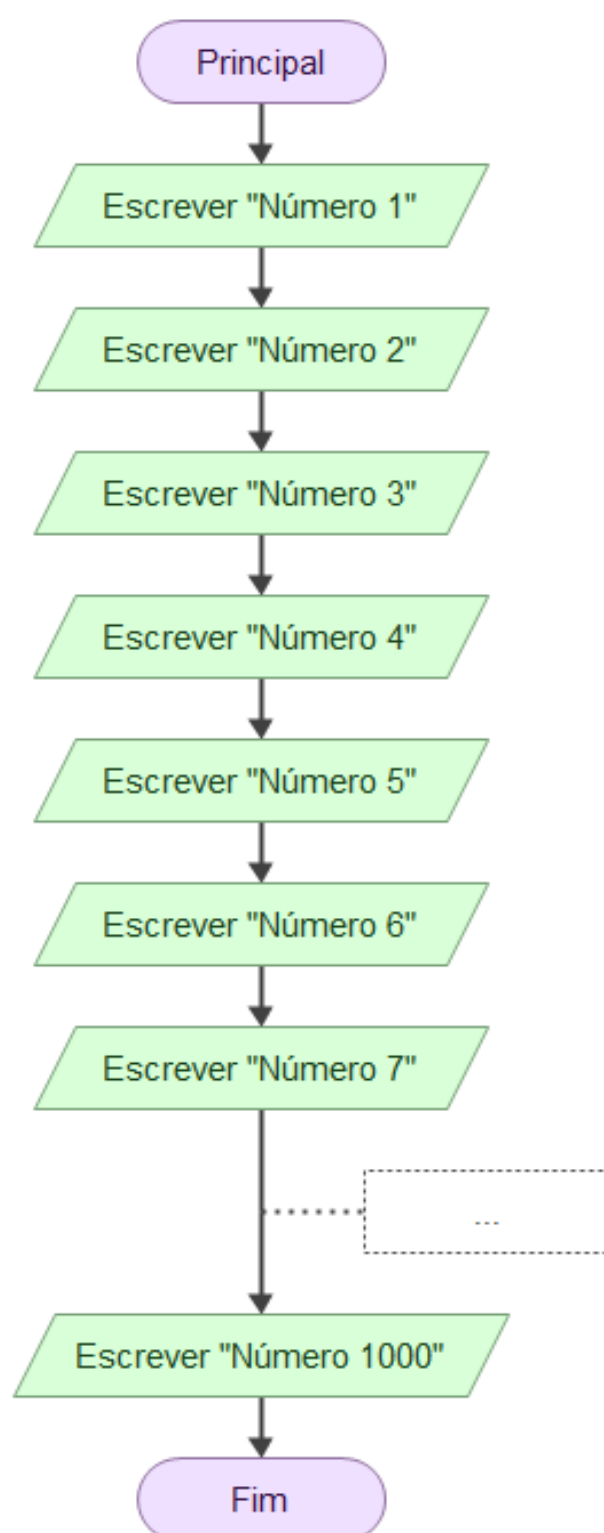
- Início;
- Definir $i = 1$
- Enquanto $i \leq 1000$:
 - Escrever "Número" + i
 - $i = i + 1$
- FimEnquanto
- Fim



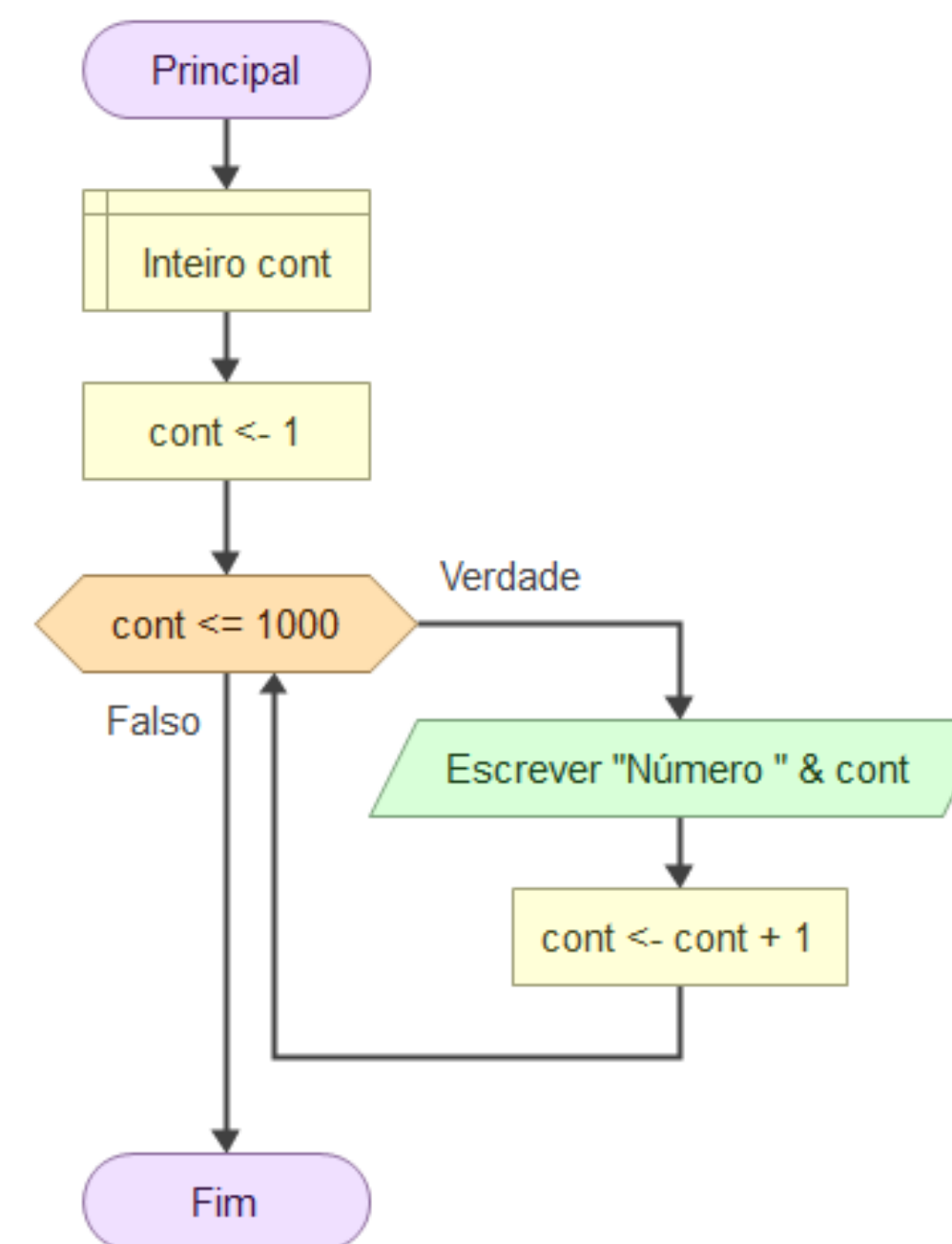
Comparação



Sem estrutura de repetição



Com estrutura de repetição

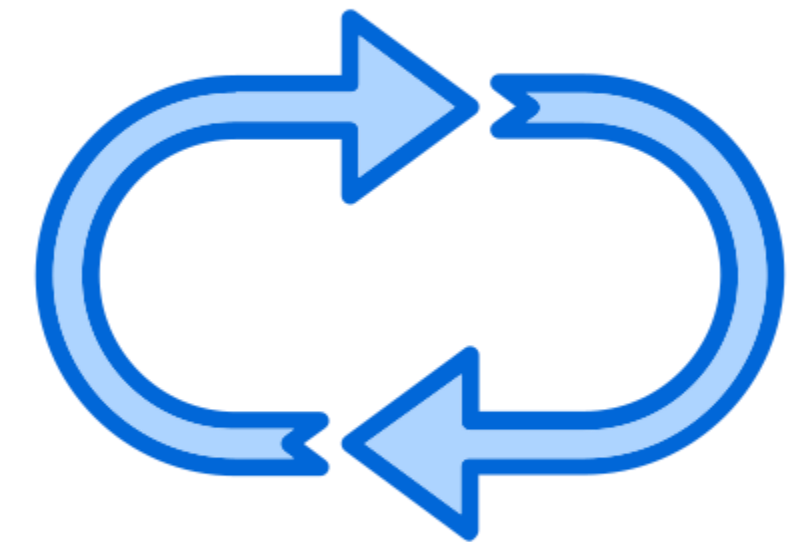


Principais estruturas de repetição

Enquanto - É a estrutura mais versátil. Ela verifica a condição antes de executar o bloco de código. Muito usado quando não se sabe exatamente quantas vezes o código vai repetir.

Fazer/Faça - Semelhante ao Enquanto, mas a verificação da condição acontece depois da execução. Usado quando se precisa que o código execute pelo menos uma vez.

Para - É uma estrutura de controle por contagem. Ela gerencia automaticamente a criação da variável, o limite e o incremento. Usado quando se sabe exatamente o número de repetições



Conceitos



Contador - É uma variável usada para contar quantas vezes uma ação foi executada dentro de uma repetição. Normalmente um contador:

- Começa com um valor inicial.
- É incrementado a cada repetição.
- Ajuda a controlar quando a repetição deve parar.
- Pode ser crescente ou decrescente.

Condição de parada - A condição de parada define o critério de interrupção da estrutura. Sem uma lógica de saída clara, ocorre o loop infinito. Essa condição é validada a cada iteração para garantir que o ciclo finalize no momento correto.

Loop Infinito - Ocorre quando a condição de parada de uma estrutura de repetição nunca é atendida (permanece sempre como verdadeira). Isso faz com que o ciclo seja executado indefinidamente, consumindo recursos do processador e podendo causar o travamento ou o congelamento do software.

Enquanto



A estrutura **ENQUANTO** é utilizada para repetir um conjunto de instruções **enquanto uma condição for verdadeira**. ou seja o programa verifica a condição primeiro, se a mesma for verdadeira, o bloco na qual a estrutura está agrupando é repetido até que seja falsa a condição.

- Antes de cada repetição, o programa **verifica a condição**.
- Se a condição for **verdadeira**, o bloco de comandos é executado.
- Se for **falsa**, a repetição termina.



Enquanto - Analogia do dia a dia



Imagine o caixa de um supermercado: o fluxo de trabalho é uma estrutura Enquanto. O funcionário atende um cliente por vez enquanto a fila não estiver vazia. A cada atendimento concluído, o número de pessoas na fila diminui (decremento). A condição de parada é atingida quando o contador de clientes chega a zero, permitindo que o funcionário encerre o ciclo de trabalho.



Atividade



Crie um algoritmo que faça a contagem de 1 a 5 e apresente cada valor.

- Início;
- Escreva "Número 1";
- Escreva "Número 2";
- Escreva "Número 3";
- Escreva "Número 4";
- Escreva "Número 5";
- Fim

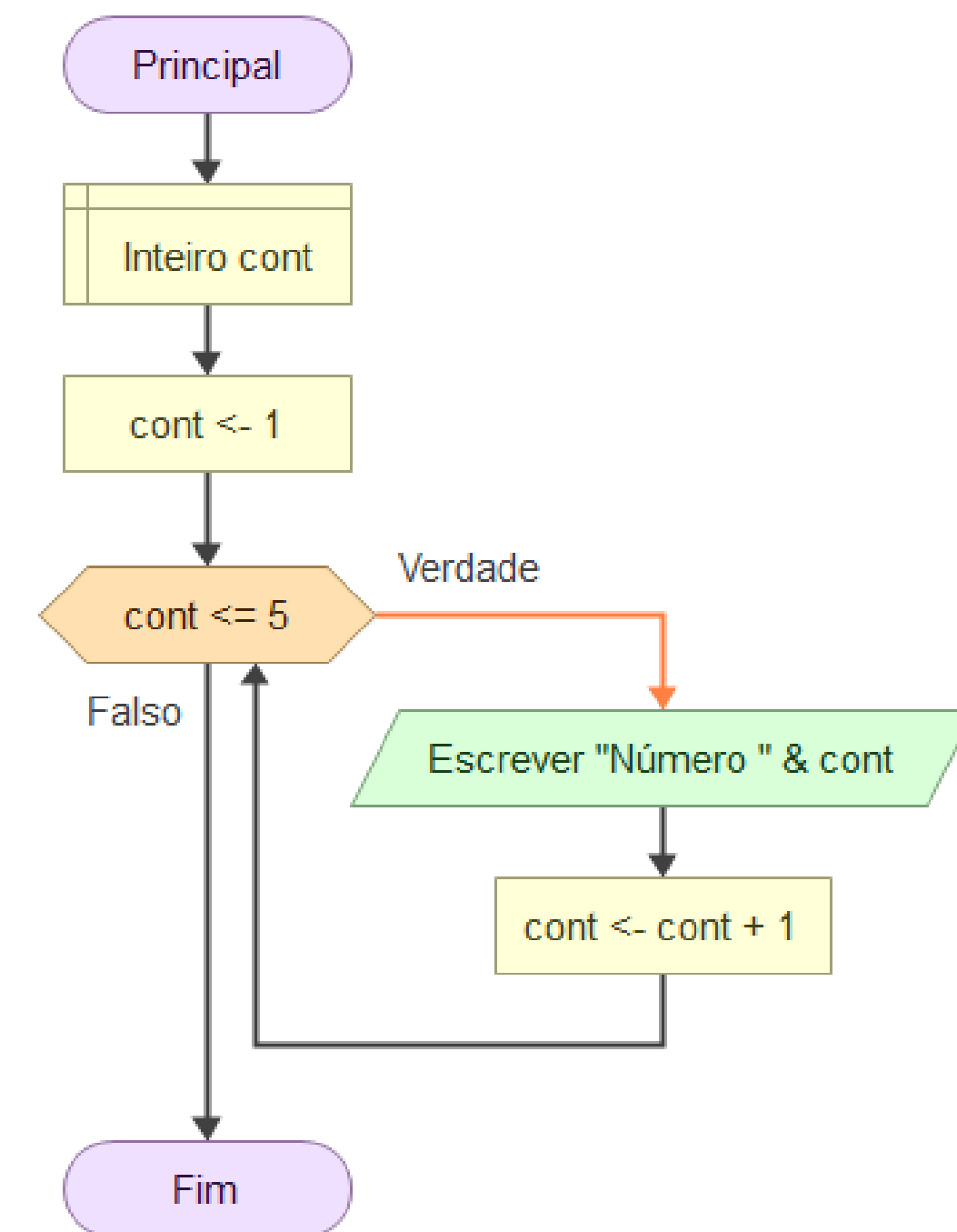
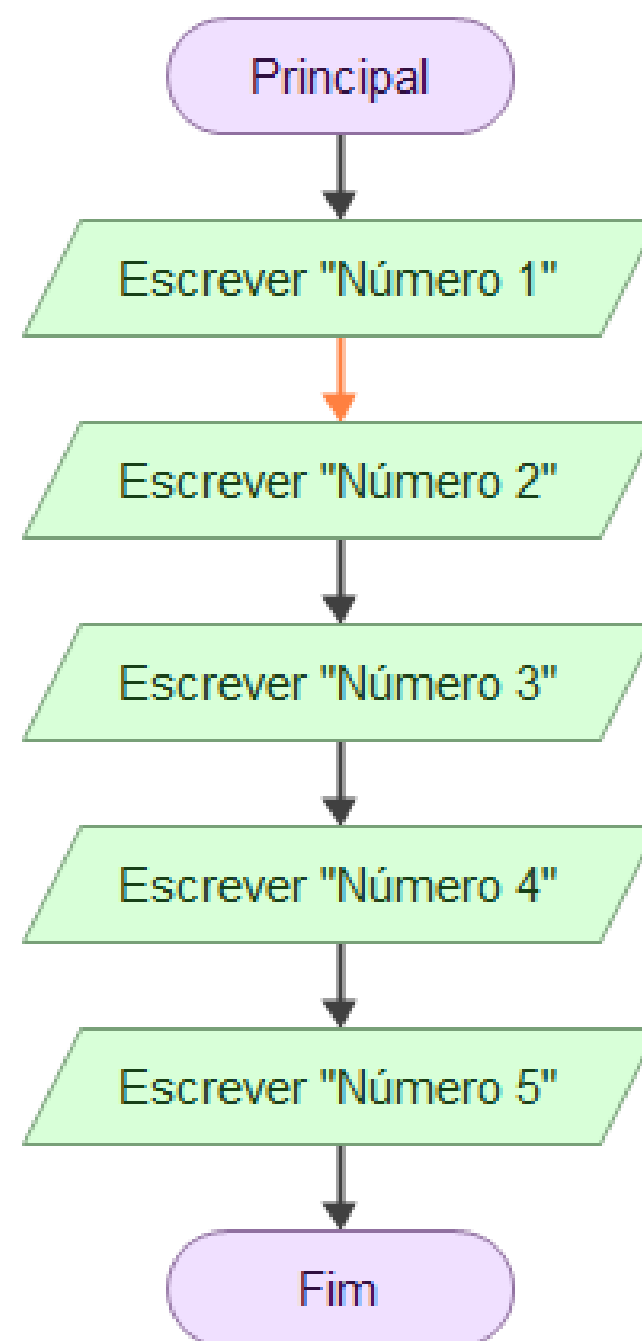
- Início;
- Definir $\text{cont} = 1$;
- Enquanto $\text{cont} \leq 5$:
 - Escrever "Número" + cont
 - $\text{cont} = \text{cont} + 1$
- FimEnquanto
- Fim



Atividade



Crie um algoritmo que faça a contagem de 1 a 5 e apresente cada valor.



Atividades



- 1 - Crie um algoritmo que mostre na tela uma contagem regressiva de 10 até 1 utilizando a estrutura ENQUANTO. Ao final da contagem, mostre a mensagem “Fim da contagem!”.
- 2 - Crie um algoritmo que leia um número inteiro informado pelo usuário e mostre todos os números **de 1 até o número informado**, utilizando a estrutura **ENQUANTO**.
- 3 - Crie um algoritmo que mostre todos os **números pares entre 1 e 100** utilizando a estrutura **ENQUANTO**.
- 4 - Crie um algoritmo que leia um número inteiro e mostre a **tabuada desse número de 1 até 10** utilizando a estrutura **ENQUANTO**. EX:
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
...



5 - Crie um algoritmo que leia um número inteiro informado pelo usuário e faça uma **contagem regressiva até 0**, utilizando a estrutura **ENQUANTO**.

Exercício Desafio - Crie um algoritmo que utilize a estrutura de repetição **ENQUANTO** para percorrer todos os números de **1 até 100**.

Durante a execução, o algoritmo deve verificar se cada número é **par** ou **ímpar**. Os números pares devem ser **somados em uma variável específica**, e os números ímpares devem ser **somados em outra variável**. Ao final da repetição, o programa deve apresentar:

- **A soma de todos os números pares entre 1 e 100**
- **A soma de todos os números ímpares entre 1 e 100**



Faça



Diferente do **enquanto**, que testa antes de executar, a principal característica dessa estrutura é que o bloco de código dentro dela é **executado pelo menos uma vez**, independentemente de a condição ser verdadeira ou falsa logo de cara.

- o bloco de comandos é executado.
- o programa **verifica a condição**.
- Se a condição for **verdadeira**, executa novamente.
- Se for **falsa**, a repetição termina.

“O enquanto pergunta antes de agir. O faça age antes de perguntar.”



Faça - Analogia do dia a dia



O atendente inicia o expediente realizando **obrigatoriamente** o primeiro atendimento do dia, independentemente de ter checado a fila antes. Somente após finalizar esse primeiro cliente e organizar o balcão é que ele olha para o salão e verifica a **condição de parada**: "*Ainda há alguém esperando?*". Se a resposta for positiva, ele repete o ciclo e atende o próximo; se a fila estiver vazia, ele encerra o trabalho.



Atividade



Crie um algoritmo que faça a contagem de 1 a 5 e apresente cada valor.

- Início;
- Escreva "Número 1";
- Escreva "Número 2";
- Escreva "Número 3";
- Escreva "Número 4";
- Escreva "Número 5";
- Fim

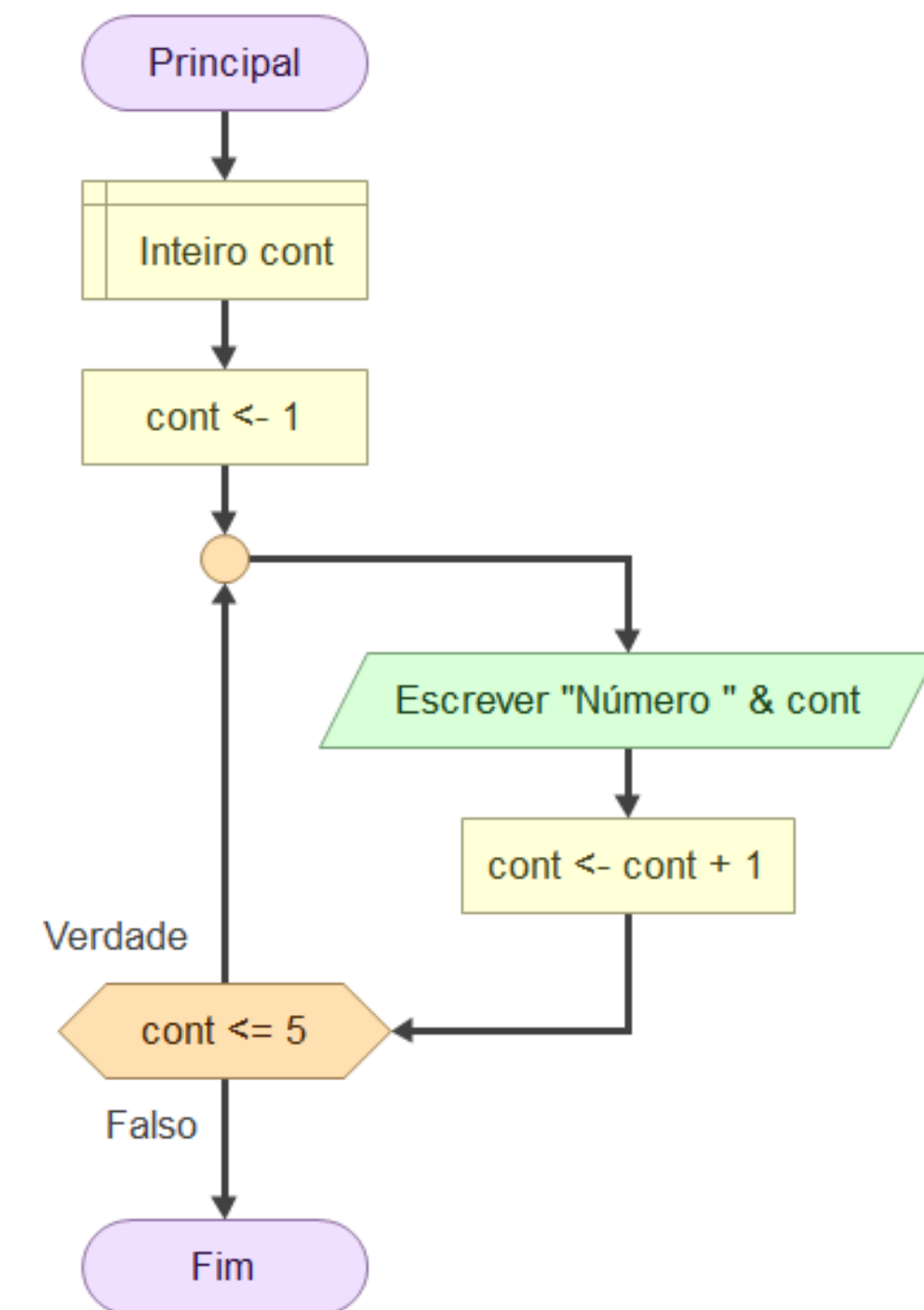
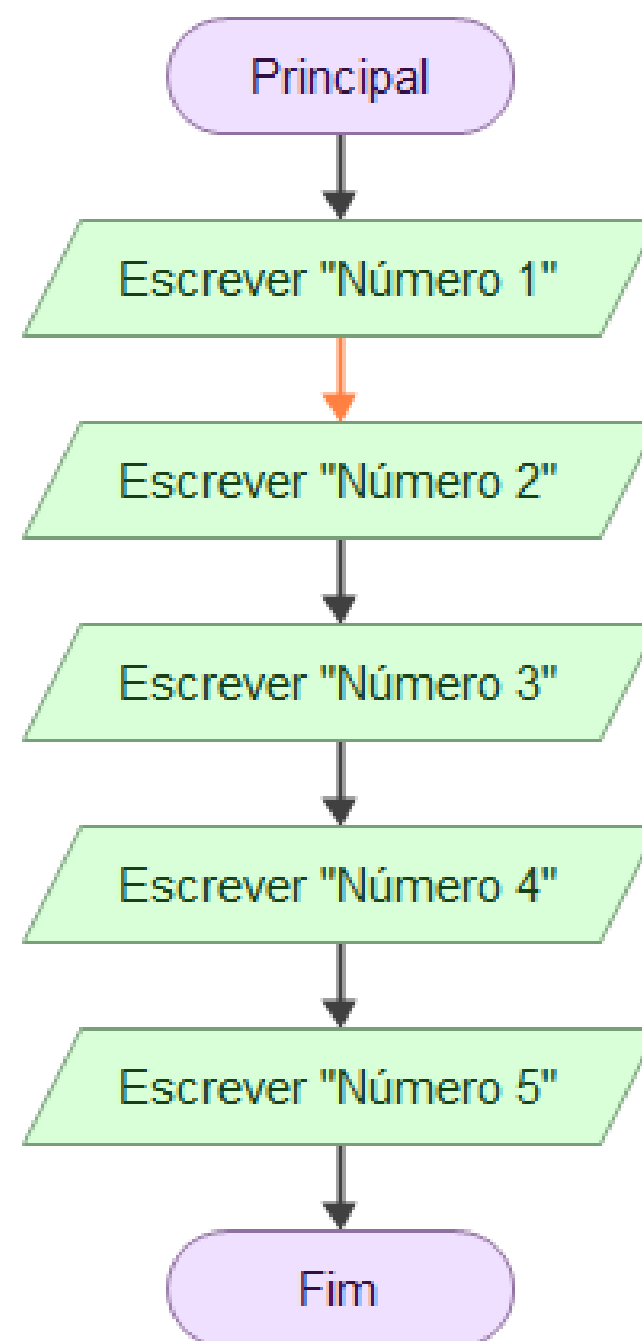
- Início;
- Definir $cont = 1$;
- Faça:
 - Escrever "Número" + $cont$;
 - $cont = cont + 1$;
 - Enquanto $cont \leq 5$;
- Fim



Atividade



Crie um algoritmo que faça a contagem de 1 a 5 e apresente cada valor.



Atividades



1 - Crie um algoritmo que peça ao usuário para digitar a senha correta.

- A senha correta é: **1234**.
- O sistema deve continuar pedindo a senha até que o usuário acerte.

2 - Crie um algoritmo que peça ao usuário um número.

- Se o número for negativo, peça novamente.
- Continue até o usuário digitar um número positivo.

3 - Crie um algoritmo que:

- Peça um número e informe o valor total atual (1ª repetição é zero)
- Some esse número a um total
- Pergunte se o usuário deseja continuar (S/N)

O programa deve parar quando o usuário responder **N** e **exibir o total novamente**.



Atividades



- 4 - Peça ao usuário para digitar uma nota entre **0 e 10**.
- Se a nota for inválida, peça novamente
 - Continue até que uma nota válida seja digitada



Para



Enquanto as outras estruturas dependem de uma condição que pode mudar a qualquer momento, o **Para** é usado quando você já sabe, antes de começar, exatamente quantas vezes o bloco de código deve ser executado.

Para a estrutura funcionar, ela precisa de três informações:

- **Início (Inicialização):** Onde o contador começa? (Ex: $i = 1$)
- **Condição (Parada):** Até onde ele vai? (Ex: $i \leq 10$)
- **Passo (Incremento):** De quanto em quanto ele pula? (Ex: de 1 em 1, ou de 2 em 2).

Para - Analogia do dia a dia



Imagine um trem que sai de uma **estação inicial** e tem um **destino final**, passando por várias estações no caminho. O funcionamento da estrutura **PARA (for)** é muito parecido com essa viagem: a **estação inicial** representa onde a repetição começa, o **destino final** indica até onde ela deve ir, e cada **parada do trem** representa uma repetição do algoritmo. A cada parada, o trem avança para a próxima estação — isso é o **incremento (passo)**. Assim, o trem segue seu percurso de forma organizada, passando por todas as estações previstas, até chegar ao destino final. Da mesma forma, o **PARA** executa um conjunto de ações um número definido de vezes, avançando passo a passo até atingir o limite estabelecido.



Atividade



Crie um algoritmo que faça a contagem de 1 a 5 e apresente cada valor.

- Início;
- Escreva "Número 1";
- Escreva "Número 2";
- Escreva "Número 3";
- Escreva "Número 4";
- Escreva "Número 5";
- Fim

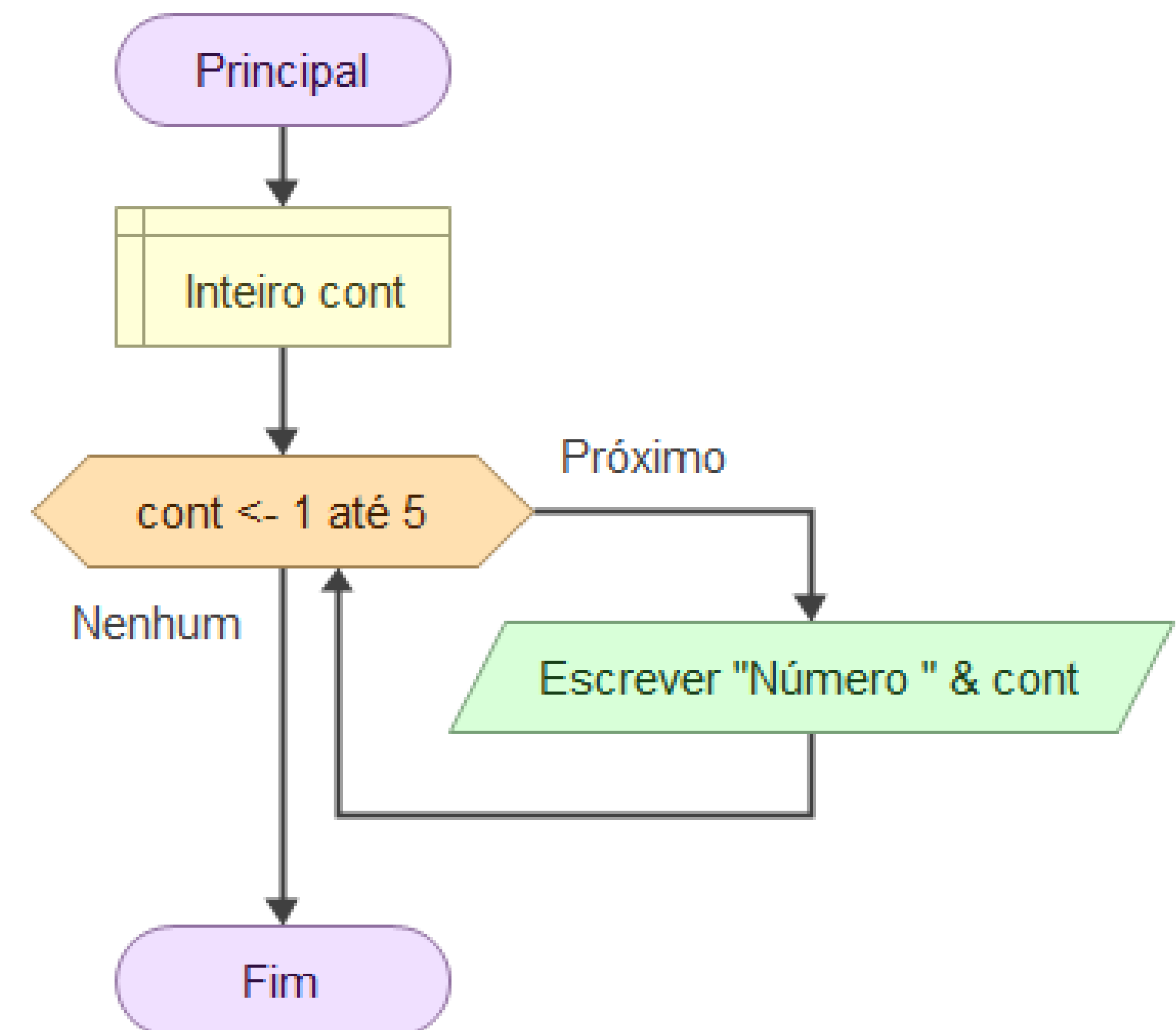
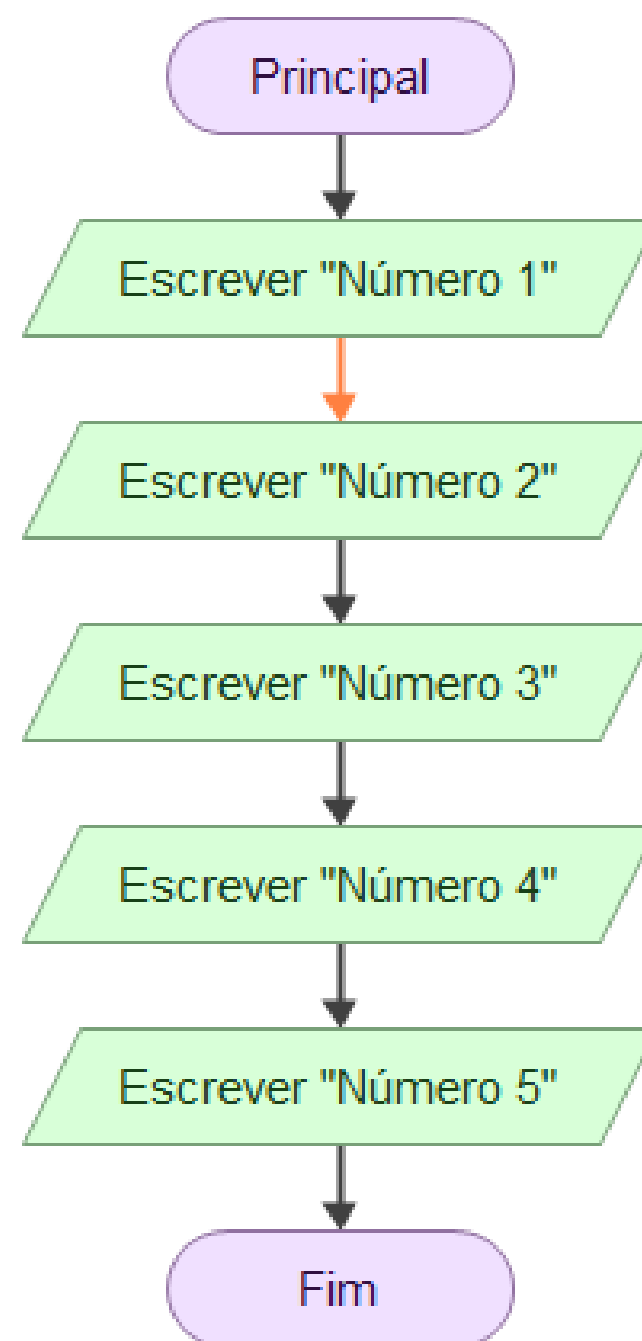
- Início;
- Definir cont = 1;
 Para cont de 1 até 5 faça
 Escrever "Número" + cont;
- FimPara
- Fim



Atividade



Crie um algoritmo que faça a contagem de 1 a 5 e apresente cada valor.



Atividades



- 1 - Crie um algoritmo que mostre os números de **1 até 10**. (utilizando a estrutura "Para")
- 2 - Mostre os números de **100 até 1**. (utilizando a estrutura "Para")
- 3 - Mostre todos os números **pares de 0 até 20**. (utilizando a estrutura "Para" e sem o operador módulos)
- 4 - Peça um número ao usuário e mostre a **tabuada desse número de 1 até 10**. (utilizando a estrutura "Para")



Atividades



5 – Um sistema precisa gerar uma sequência personalizada. Para isso solicite dois números ao usuário um de início e outro de fim. Mostre todos os números entre eles.

6 - Peça um número e calcule o **fatorial** dele.

Exemplo:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = \mathbf{120}$$

Desafio:

Faça o tratamento, solicitando que seja inserido novamente um valor, caso:

- **Número negativo (não existe fatorial)**
- **Fatorial de 0 ($0! = 1$)**



Estruturas de dados



Estruturas de Dados são formas organizadas de armazenar, gerenciar e manipular dados em um computador para que eles possam ser acessados e modificados de forma eficiente.

Não existe uma estrutura "perfeita", existe a estrutura certa para o problema certo. Elas são como ferramentas em uma caixa: você não usa um martelo para parafusar algo, mas isso não faz o martelo ruim.

Temos diversas estruturas de dados, mas neste momento vamos focar em duas: vetores e matrizes.



O que é um Vetor (Array)?



Imagine uma **fileira** de armários em um vestiário ou uma escola.

- Todos os armários são do mesmo tamanho.
- Todos estão grudados um no outro, em sequência.
- Cada armário tem um número (índice) na porta para você saber qual é qual.

Na computação, um Vetor é uma estrutura que armazena uma sequência de elementos do mesmo tipo (todos números, ou todos nomes, etc.) em posições de memória que ficam uma ao lado da outra.



Características



Para entender um vetor, você precisa entender estas três características:

1. O Índice começa no ZERO

O computador não começa a contar do 1, mas sim do 0:

O 1º armário é o índice 0.

O 2º armário é o índice 1.

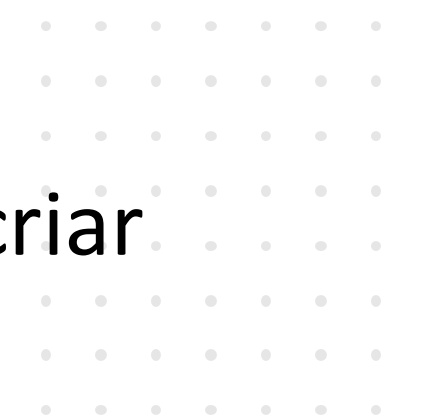
Se o vetor tem 10 espaços, o último é o índice 9.

2. Tipo Único (Homogeneidade)

Um vetor é como uma caixa de ovos: você só coloca ovos dentro. Você não mistura um número inteiro com uma frase (string) no mesmo vetor.

3. Tamanho Fixo (Estático)

Normalmente, quando você cria um vetor, você diz: "Quero um vetor de 5 posições". O computador reserva esse espaço e pronto. Se você precisar de um 6º espaço, vai ter que criar um vetor novo e maior.



Array



Índice

0

1

2

3

4

Valor

--	--	--	--	--



Como declarar?



Imagine:

--	--	--

Declaração - Propriedades

Declaração

A instrução de Declaração permite para criar as variáveis utilizadas para guardar dados durante a execução do programa.

Nomes de variáveis:

numeros

Tipo:

☒ Inteiro

☐ Real

☐ Texto

☐ Booleano

☒ Matriz?

Dimensão da matriz:

3

OK

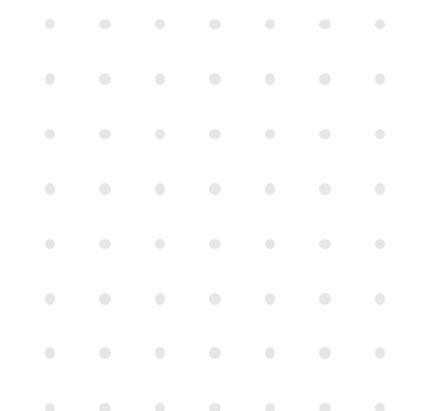
Cancelar

Matrizes



Enquanto o vetor tem apenas uma dimensão (uma linha), a matriz é **bidimensional**: ela tem **linhas** e **colunas**. É exatamente como uma planilha do Excel ou um tabuleiro de xadrez.

	A	B	C	D	E
1					
2					
3					
4					
5					
6					
7					
8					



Matrizes



Uma matriz é um conjunto de vetores empilhados. Para encontrar um dado dentro dela, você **não precisa mais de apenas um endereço (índice), mas de dois (linha, coluna):**

- O número da **Linha**.
- O número da **Coluna**.

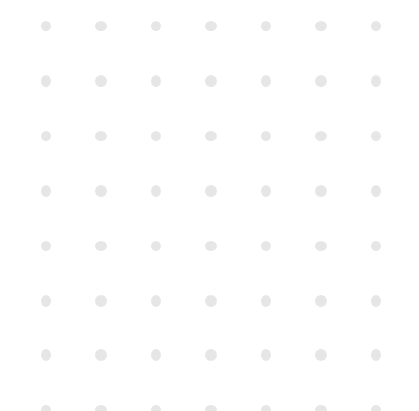
	A	B	C
1	5	12	15
2	37	58	8
3	98	91	1

Pense na matriz ao lado (3x3):

Na posição (1,A) qual o valor?

Na posição (3,C) qual o valor?

Na posição (3,A) qual o valor?



Atividades



1 - Crie um algoritmo que:

- Declare um vetor com capacidade para armazenar **10 números inteiros**
- Solicite ao usuário que digite os valores
- Ao final, exiba todos os números informados na ordem inversa em que foram digitados

2 - Desenvolva um algoritmo que:

- Armazene **8 números inteiros** em um vetor
- Calcule a soma de todos os elementos
- Exiba o resultado final

3 - Crie um algoritmo que:

- Armazene as notas de **5 alunos** em um vetor
- Calcule a média das notas
- Exiba a média final



Atividades



4 - Desenvolva um algoritmo que:

- Leia **10 números inteiros** e armazene em um vetor
- Identifique e exiba:
 - O **maior valor**
 - O **menor valor**

5 - Crie um algoritmo que:

- Armazene **10 números inteiros**
- Conte quantos são:
 - Pares
 - Ímpares
- Exiba os totais



Atividades



Desafio - Crie um algoritmo que:

- Declare um vetor com capacidade para armazenar **10 números inteiros**
- Solicite ao usuário que preencha o vetor

Após o preenchimento, o algoritmo deve:

- Exibir os valores na ordem original
- Ordenar os valores em **ordem crescente** (do menor para o maior)
- Exibir o vetor ordenado



A nighttime photograph of a city street, likely in São Paulo, Brazil, featuring light trails from cars and a prominent SENAIA logo overlay. The scene is dark, with city lights and building facades visible in the background. The logo is a red rectangle with the word "SENAIA" in white, bold, sans-serif font. The background shows a wide street with a crosswalk, a tram stop on the right, and various buildings, including a large one with arched windows on the left and a tall, modern one on the right. Light trails from cars are visible on the road, and a yellow construction crane is visible in the distance.

SENAIA

Obrigado